

Capture Screenshots, Videos & Trace Files in Playwright

Introduction

While executing automation tests, it is often useful to capture additional information for debugging failures.

Playwright provides three powerful debugging features:

-  Screenshots
-  Videos
-  Trace Files

These artifacts help you quickly identify what happened during test execution without rerunning the test.

1. Capture Screenshots

A screenshot captures the current state of the browser page.

It is especially useful when a test fails because you can see exactly what appeared on the screen.

There are 2 methods to capture screenshots

Method 1: Programmatically

Use the `page.screenshot()` method anywhere inside your test.

```
import { test } from '@playwright/test';

test('Capture Screenshot', async ({ page }) => {

  await page.goto('https://example.com');

  await page.screenshot({
    path: 'screenshots/homepage.png'
  });

});
```

Capture Full Page Screenshot

```
await page.screenshot({
  path: 'screenshots/fullpage.png',
  fullPage: true
});
```

Common Screenshot Options

Option	Description
path	File location
fullPage	Captures the entire page
omitBackground	Removes page background
quality	Image quality (JPEG only)
type	png or jpeg

Example:

```
await page.screenshot({
  path: 'homepage.jpg',
  type: 'jpeg',
  quality: 90
});
```

Method 2: Using `playwright.config.ts`

Instead of writing screenshot code inside every test, Playwright can automatically capture screenshots.

```
use: {  
  
  screenshot: 'only-on-failure'  
  
}
```

Screenshot Options

```
screenshot: 'off'
```

No screenshots.

```
screenshot: 'on'
```

Capture screenshots for every test.

```
screenshot: 'only-on-failure'
```

Capture screenshots only when a test fails.

This is the most commonly used option.

Where are screenshots stored?

Screenshots are automatically stored inside

```
test-results/
```

Example:

```
test-results/  
  
  Login-Test/  
    test-failed-1.png
```

2. Capture Videos

Videos record the complete execution of a test.

They are extremely useful for debugging intermittent or UI-related failures.

There are 2 methods

Method 1: Programmatically via Browser Context

Videos are enabled while creating a Browser Context.

```
import { chromium } from '@playwright/test';  
  
const browser = await chromium.launch();  
  
const context = await browser.newContext({  
  
  recordVideo: {  
    dir: 'videos/'  
  }  
  
});  
  
const page = await context.newPage();
```

After the test finishes:

```
await context.close();
```

The video is automatically saved.

Method 2: Globally using `playwright.config.ts`

S

```
use: {  
  
  video: 'on-first-retry'  
  
}
```

Video Options

Disable videos

```
video: 'off'
```

Record every test

```
video: 'on'
```

Keep videos only for failed tests

```
video: 'retain-on-failure'
```

This is the most commonly used option.

Record only the first retry

```
video: 'on-first-retry'
```

Useful when retrying flaky tests.

Video Options Summary

Option	Description
off	No videos
on	Record every test
retain-on-failure	Keep videos only for failed tests
on-first-retry	Record only the first retry

Where are videos stored?

Videos are automatically stored inside

```
test-results/
```

Example

```
test-results/  
  
  Login-Test/  
    video.webm
```

Note: Videos are saved in the `.webm` format.

3. Capture Trace Files

A Trace File is one of Playwright's most powerful debugging features.

A trace records almost everything that happened during a test.

It includes:

- Every action performed
- Screenshots
- DOM snapshots

- Console logs
- Network requests
- Timing information
- Source code (optional)

Think of a trace as a "video + screenshots + browser developer tools" combined into one file.

There are 3 methods

Method 1: Using `playwright.config.ts`

```
use: {  
  
  trace: 'on'  
  
}
```

Trace Options

```
trace: 'off'
```

Disable tracing.

```
trace: 'on'
```

Record every test.

```
trace: 'retain-on-failure'
```

Keep traces only for failed tests.

Recommended for most projects.

```
trace: 'on-first-retry'
```

Record trace only during the first retry.

Trace Options Summary

Option	Description
off	Disable tracing
on	Record every test
retain-on-failure	Keep traces only for failed tests
on-first-retry	Record only first retry

Method 2: Using CLI Command

You can enable tracing while executing tests.

```
npx playwright test mytest.spec.ts --trace on
```

Other examples

```
npx playwright test --trace on
```

```
npx playwright test --trace retain-on-failure
```

Method 3: Programmatically

Start tracing

```
await context.tracing.start({

  screenshots: true,
  snapshots: true,
  sources: true

});
```

Execute your test

```
await page.goto('https://example.com');

// Test steps...
```

Stop tracing

```
await context.tracing.stop({

  path: 'traces/playwright-trace.zip'

});
```

Tracing Options

Option	Description
screenshots	Captures screenshots after actions
snapshots	Captures DOM snapshots
sources	Includes your TypeScript/JavaScript source code in the trace

Trace File Location

Example

```
traces/

  playwright-trace.zip
```

or

```
test-results/  
  
    trace.zip
```

How to View a Trace File

There are **3 ways** to open a trace.

Method 1: From HTML Report

Generate the HTML report

```
npx playwright show-report
```

Open the report in the browser.

Click

```
trace.zip
```

Playwright Trace Viewer opens automatically.

Method 2: Using CLI

```
npx playwright show-trace traces/playwright-trace.zip
```

The Trace Viewer opens immediately.

Method 3: Using the Online Trace Viewer

Open:

<https://trace.playwright.dev/>

Then:

- Drag and drop `trace.zip`, or
- Upload the file.

The trace opens directly in your browser.

What Can You See Inside a Trace?

The Trace Viewer provides:

- Timeline of all actions
- Screenshots after each action
- DOM snapshots
- Console logs
- Network requests
- Source code
- Action timings
- Error messages

It is one of the best tools for debugging failed Playwright tests.

Handle Flaky Tests with Retries

What is a Flaky Test?

A flaky test is a test that produces inconsistent results.

Sometimes it passes.

Sometimes it fails.

Even though the application has not changed.

Example reasons:

- Slow network
- Slow page loading
- Timing issues
- Dynamic UI updates
- Third-party service delays

Instead of failing immediately, Playwright can retry the test.

3 Ways to Configure Retries

1. Global Configuration

Configure retries for the entire project.

```
// playwright.config.ts

export default defineConfig({

  retries: 2

});
```

Every failed test will be retried up to **2 times**.

2. Command Line Flag

Override the retry count while running tests.

```
npx playwright test --retries=2
```

Useful when you want retries only for a particular execution.

3. Using `test.describe.configure()`

Configure retries for a specific test suite.

```
import { test } from '@playwright/test';

test.describe.configure({
  retries: 2
});

test.describe('Login Tests', () => {

  test('Login Test', async ({ page }) => {

    // Test steps

  });

});
```

Only the tests inside this `describe` block will use retries.

Retry Execution Flow

Suppose:

```
retries = 2
```

Execution flow:

```
Attempt 1
  ↓
Fail
  ↓
Retry 1
  ↓
Fail
  ↓
Retry 2
  ↓
Pass or Fail
```

Maximum attempts:

Original Test + 2 Retries = 3 Executions



Best Practices

- Use `screenshot: 'only-on-failure'` to avoid unnecessary screenshots.
- Use `video: 'retain-on-failure'` to save storage while preserving failed test recordings.
- Use `trace: 'retain-on-failure'` because trace files provide the most detailed debugging information.
- Enable retries only for genuinely flaky tests; do not use retries to hide real defects.
- Review screenshots, videos, and traces together to quickly identify the root cause of failures.
- Clean up old `test-results/` and trace files periodically to save disk space.



Summary

Feature	Purpose	Recommended Setting
Screenshot	Capture browser state	<code>only-on-failure</code>
Video	Record test execution	<code>retain-on-failure</code>
Trace	Complete debugging information	<code>retain-on-failure</code>
Retries	Re-run flaky tests automatically	<code>retries: 1</code> or <code>2</code> (only when needed)

These features make Playwright one of the most powerful automation frameworks for debugging UI tests. By combining screenshots, videos, trace files, and retries, you can quickly investigate failures, reduce debugging time, and improve the reliability of your automated test suite.